

Prouver l'exécution d'un automate (STARK)

Mathéo Le Dévéhat

23112

2025-2026

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions
- 4 AIR
- 5 FRI
- 6 Récapitulatif
- 7 Conclusion

1 Objectif et motivations

2 Prérequis

3 Définitions

4 AIR

5 FRI

6 Récapitulatif

7 Conclusion

Objectif

Sachant un automate A , prouver qu'on connaît un mot w tel que l'automate reconnaisse ce mot sans le divulguer.

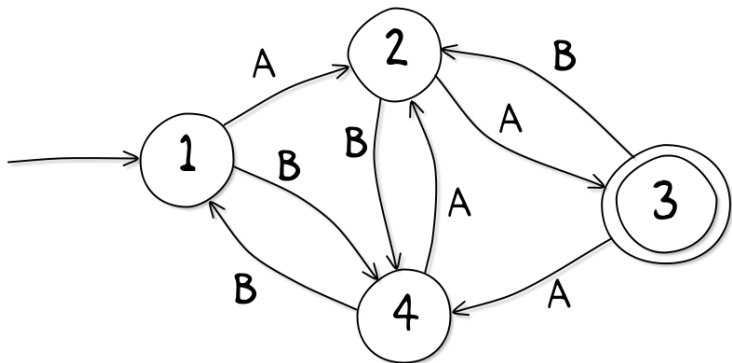


Figure: Un exemple d'automate (csfieldguide.org.nz)

Permet de prouver son identité sans révéler des informations supplémentaires.

- 1 Objectif et motivations
- 2 Prérequis**
- 3 Définitions
- 4 AIR
- 5 FRI
- 6 Récapitulatif
- 7 Conclusion

Prérequis: Fonction de hachage

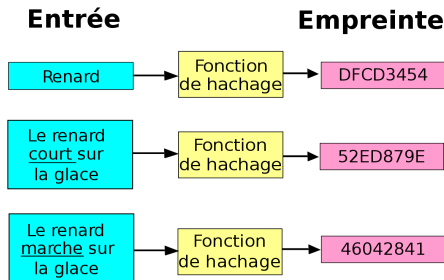


Figure: Illustration d'une fonction de hachage (techno-science.net)

Critères essentiels:

- 1 Résistance aux collisions
- 2 Résistance à la préimage
- 3 Résistance à la deuxième préimage
- 4 Distribution uniforme

Prérequis: Arbres de Merkle

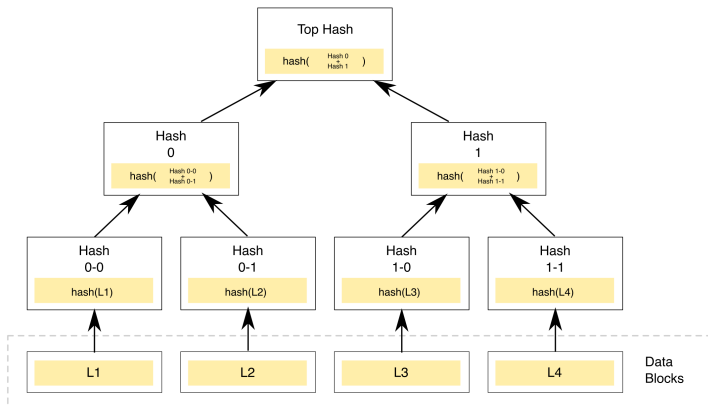
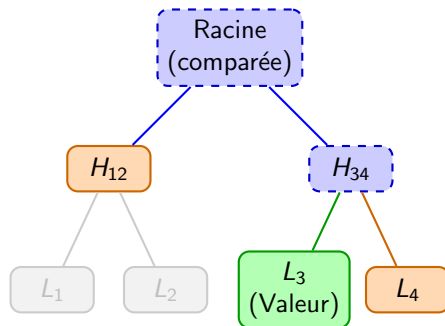


Figure: Un exemple d'arbre de Merkle ([wikipedia.org](https://fr.wikipedia.org/wiki/Arbre_de_Merkle))

Attestation d'une valeur spécifique

Le Chemin d'Authentification

- Pour prouver qu'une feuille (**Valeur** L_3) appartient à l'arbre, on ne transmet pas tout l'arbre.
- On fournit uniquement les **nœuds frères** sur le chemin (**nœuds oranges**).
- Le vérificateur recalcule les parents itérativement (**nœuds bleus**) jusqu'à la racine.



Efficacité : La taille de la preuve n'est que de $\mathcal{O}(\log_2(N))$ éléments, rendant les STARKs très compacts pour de grandes traces d'exécution.

Formule de reconstruction

$$H_{\text{parent}} = \text{Hash}(H_{\text{gauche}} \parallel H_{\text{droite}})$$

Prérequis: L'heuristique de Fiat-Shamir

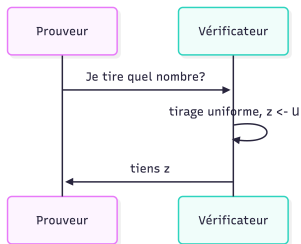


Figure: Version interactive

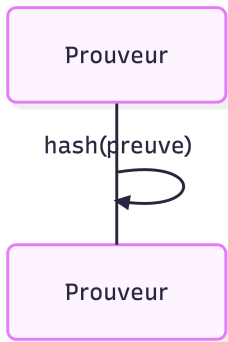


Figure: Version non interactive

Sommaire

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions**
- 4 AIR
- 5 FRI
- 6 Récapitulatif
- 7 Conclusion

Définition ZK-STARK

Zero-Knowledge, Succint, Transparent, Argument of Knowledge. En français, argument succinct et transparent de connaissance à divulgation nulle de connaissance, ie. prouver qu'on connaît un certain w (témoin) satisfaisant une propriété P sans divulguer w .

Une Architecture de ZK-STARK se compose de deux entités, le prouveur et le vérificateur. Généralement le prouver est une entité centrale qui a une grosse puissance de calcul et des données qu'elle souhaite garder privées et le vérificateur est accessible publiquement et à une puissance de calcul moindre. Cas d'usage: vérification de programmes lourds sur une blockchain publique.

Prouver l'exécution d'un automate

Déclaration naïve

Je connais un mot W accepté par un automate A

Les variables en majuscule sont publiques et les variables en minuscules ne sont accessibles au prouveur. Pour prouver cela, l'idée est de prouver que chaque transition de l'automate est correcte.

Sommaire

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions
- 4 AIR**
- 5 FRI
- 6 Récapitulatif
- 7 Conclusion

AIR - Algebraic Intermediate Representation

Une AIR est un moyen de transformer un problème de calcul en problème algébrique. On veut donc prouver que les transitions entre chaque états de l'automate sont valides.

étape	état	caractère
0	1	B
1	4	B
2	1	A
3	2	B
4	4	A
5	2	ε

Table: Table de transitions

On va utiliser des polynômes. Plus précisément un polynôme par colonne. Donc un polynôme étape P_i , état P_e et caractère P_c . Pour cela on utilise l'interpolation de Lagrange. Par exemple:

$$P_e = \frac{11x^5}{120} - \frac{35x^4}{24} + \frac{65x^3}{8} - \frac{445x^2}{24} + \frac{887x}{60} + 1$$

On veut maintenant prouver que notre exécution, donc nos polynômes respectent les 3 contraintes suivantes:

- 1 L'exécution démarre à l'état initial
- 2 Chaque transition est valide (est dans la définition de l'automate donné en paramètre)
- 3 L'exécution termine sur un état final

On doit transformer ces contraintes en polynômes. Comme on a la propriété suivante:

$$P(x) = 0 \iff (X - x) \mid P \iff \frac{P}{X - x} \text{ est un polynôme}$$

Nos contraintes deviennent:

$$\begin{aligned} \textcircled{1} \quad C_0 &= \frac{P_e(0) - q_0}{X} \\ \textcircled{2} \quad C_1 &= \frac{P_e(X+1) - P_t(P_e(X), P_c(X))}{\prod_{0 \leq i \leq n-1} (X-i)} \\ \textcircled{3} \quad C_2 &= \frac{\prod_{q \in F} P_e(n-1) - q}{X - (n-1)} \end{aligned}$$

Avec P_t le polynôme à deux variables de transitions valides qui à un état de départ et un caractère associe un unique état de destination.

On veut maintenant appliquer le protocole FRI. Pour des raisons de simplicité, on va se ramener à un seul polynôme pour cela, on va créer le polynôme de composition (CP) suivant:

$$CP = \alpha_0 C_0 + \alpha_1 C_1 + \alpha_2 C_2$$

Avec $\alpha_0, \alpha_1, \alpha_2$ des coefficients tirés aléatoirement de manière uniforme par le vérificateur. Ils servent à éviter que le prouveur ne triche, c'est-à-dire à garder l'équivalence

$CP(x) = 0 \iff$ en probabilités $C_0(x) = 0$ et $C_1(x) = 0$ et $C_2(x) = 0$. On peut prouver que cela fonctionne même si le vérificateur "triche", c'est-à-dire qu'il choisit les coefficients.

Sommaire

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions
- 4 AIR
- 5 FRI**
- 6 Récapitulatif
- 7 Conclusion

Protocole FRI: Fast Reed-Solomon Interactive Oracle Proof of Proximity

On veut maintenant montrer que notre polynôme de composition s'annule sur toutes les étapes de calcul. Pour cela, on pourrait de manière naïve, envoyer le polynôme au vérificateur et l'ensemble des points. Mais on perd tout l'intérêt: pas de compression, pas de données privées.

Objectif

Prouver que CP est un polynôme. Avec FRI on va se ramener à un polynôme de degré 1. L'opérateur FRI nous permet de se ramener à un polynôme de degré $D/2$. On le répète jusqu'à atteindre un polynôme constant.

$$\Delta(f, P) = \frac{|\{x \in D \mid f(x) \neq P(x)\}|}{|D|}$$

Plus théoriquement: On veut montrer qu'avec une certaine probabilité, notre fonction est δ proche d'un polynôme: $P(\Delta(f, P) < \delta) \approx 1$. Et l'opérateur FRI permet de diviser le degré d'un polynôme par 2, sans perdre sa propriété de δ proche. Ainsi le vérificateur a uniquement besoin de vérifier qu'une série de points est une fonction affine.

Comment fonctionne FRI

Inspiration

L'idée est la même que l'opérateur FFT pour les polynômes.

- 1 On sépare le polynôme en un polynôme avec les puissances paires, et un autre avec les impaires $P(X) = p(X^2) + Xi(X^2)$
- 2 On tire un β aléatoirement (Fiat-Shamir)
- 3 On considère maintenant $\tilde{P} = p(X) + \beta i(X)$

Commitment

Il faut attester chacune des applications de l'opérateur FRI.

Sommaire

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions
- 4 AIR
- 5 FRI
- 6 Récapitulatif**
- 7 Conclusion

Voici ce que connaît le vérificateur:

- 1 l'engagement de CP
- 2 l'engagement de chaque application de FRI
- 3 tous les tirages aléatoires

- 1 Générer la trace de l'exécution de l'automate et de la fonction de hachage
- 2 Créer tous les polynômes de représentation intermédiaires
- 3 Évaluer ces polynômes et les engager
- 4 Créer CP (donc vérifier les contraintes)
- 5 Appliquer FRI sur CP itérativement
- 6 Répéter la dernière étape autant de fois qu'on veut (nombre de points d'évaluation)
- 7 Envoyer des métadonnées, les engagements, preuves d'engagements au vérificateur

Idée

L'idée est de refaire le processus inverse qu'à fait le prouver mais au lieu de calculer les polynômes on vérifie que les évaluations correspondent.

- ❶ Récupérer les métadonnées, engagement et preuves d'engagements
- ❷ Pour chaque point d'évaluation
 - ❶ Vérifier l'engagement pour l'état, le caractère, et la transition
 - ❷ Vérifier les engagements du hash
 - ❸ Reconstruire **l'évaluation** de CP
 - ❹ Pour chaque application de FRI
 - ❶ Vérifier les engagements
 - ❷ Appliquer FRI au point
 - ❸ Vérifier que le résultat est correct

Prouver l'exécution d'un automate

Déclaration correcte

Je connais un mot w accepté par un automate A tel que $h(w) = H$

Les variables en majuscule sont publiques et les variables en minuscules ne sont accessibles au prouveur.

Poséidon: une fonction de hachage "ZK-friendly"

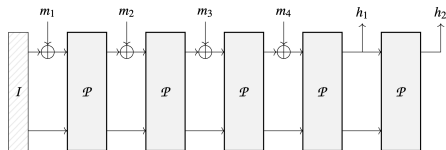


Figure: Une fonction de hachage en éponge

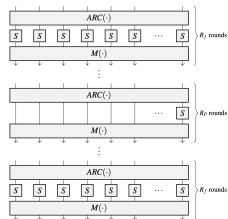


Figure: La permutation poséidon

Sommaire

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions
- 4 AIR
- 5 FRI
- 6 Récapitulatif
- 7 Conclusion**

- ① 01-02/25: Lecture des papiers sur Cairo, STARK et FRI
- ② 03/25: STARK 101
- ③ 04-06/25: Polynômes et utilitaires en Ocaml
- ④ 09/25: Idée AFD et premier prototype d'AIR
- ⑤ 11/25: Prototype fini sans ZK
- ⑥ 01/26: Fini la rédaction d'articles sur ce premier prototype
- ⑦ 02/26: Fini prototype avec le hash
- ⑧ 02/26: Gros problème

Annexe: Code

```
1  CONSTRAINTS_LENGTH=29
2  N_QUERY=12
3  T=8
4  r_f = 8
5  r_p = 57
6  blowup_factor = 128 # WARNING: needs to be a power of two
```

Annexe: Code

```
136
137 ✓ class Sponge():
138 ✓     def __init__(self, rate, capacity, permutation, r_f, r_p, rc):
139         self.r = rate
140         self.c = capacity
141         self.pi = permutation
142         self.state = None
143         self.r_f = r_f
144         self.r_p = r_p
145         self.mds = generate_cauchy_matrix(rate+capacity)
146         self.rc = ARC
147         self.trace = []
148
149 ✓     def apply(self, data):
150         self.state = np.array([FieldElement(0) for i in range(0, self.c+self.r)])
151         self.trace = [self.state]
152         #remaining = self.pad(remaining)
153         chunks = [data[self.r*i:self.r*(i+1)] for i in range(0, len(data)//self.r)]
154         i = 0
155         while(len(chunks) > 0):
156             i+=1
157             self.state, trace = self.pi(self.state + np.pad(chunks[0], (0, self.c), mode='constant'), self.r_f, self.r_p, self.mds, self.rc)
158             chunks = chunks[1:]
159             self.trace.extend(trace)
160         return self.state[0:self.c], self.trace
161
162
163     def generate_cauchy_matrix(t):
164         x = [FieldElement(e) for e in range(0, t)]
165         y = [FieldElement(e) for e in range(0, 2*t)]
166         return np.array([(FieldElement(1)/(x[i]-y[j]) for j in range(0, len(y))) for i in range(0, len(x))])
167
168     def arc(state, rc):
169         return np.array(state) + rc
170
171     def mix(state, mds):
172         return mds @ np.array(state)
173
174     def full_s_box(state):
175         return np.array(state) ** 5
176
177     def partial_s_box(state):
178         return [state[0] ** 5, *state[1:]]
179
```


Annexe: Code

```
180 ✓ def hades_round_permutation(state, r_f, r_p, mds, rc):
181     trace = []
182     for r in range(0, r_f+r_p):
183         state = arc(state, rc[r])
184         if(r_f/2 <= r < r_f/2+r_p):
185             state = partial_s_box(state)
186         else:
187             state = full_s_box(state)
188         state = mix(state, mds)
189         trace.append(state)
190     return state, trace
191
192 ✓ class Poseidon():
193     def __init__(self):
194         self.sponge = Sponge(7,1,hades_round_permutation,8,67, ARC)
195
196     def hash(self, data):
197         return self.sponge.apply(data)
```

Annexe: Code

```
1 import json
2 import math
3
4 # Source - https://stackoverflow.com/a/3035188
5 # Posted by Robert William Hanks, modified by community. See post 'Timeline' for change history
6 # Retrieved 2020-02-02, License - CC BY-SA 4.0
7
8 def primes(n):
9     """ Returns a list of primes < n """
10    sieve = [True] * n
11    for i in range(3, int(n**0.5)+1, 2):
12        if sieve[i]:
13            sieve[i::2*i] = [False]*((n-i-1)//(2*i)+1)
14    return [2] + [i for i in range(3, n, 2) if sieve[i]]
15
16 # =====
17
18 class Automata:
19     def __init__(self, states, alphabet, transition, start_state, accept_states):
20         self.states = sorted(list(states))
21         self.alphabet = sorted(list(alphabet))
22         self.transition = transition
23         self.start_state = start_state
24         self.accept_states = accept_states
25         n_needed = len(self.states) + len(self.alphabet)
26         safe_bound = int(n_needed * (math.log(n_needed) + 4)) + 20 if n_needed > 2 else 20
27         p_list = primes(safe_bound)[:n_needed]
28
29         # 2. Construct numeric maps
30         self.state_map = {state: p_list[i] for i, state in enumerate(self.states)}
31         self.character_map = {char: p_list[len(self.states) + i] for i, char in enumerate(self.alphabet)}
32
33     @classmethod
34     def from_json(cls, json_string):
35         """Creates an instance of Automata from a JSON string."""
36         data = json.loads(json_string)
37         return cls(
38             states=set(data["states"]),
39             alphabet=set(data["alphabet"]),
40             transition=data["transition"],
41             start_state=data["start_state"],
42             accept_states=set(data["accept_states"])
43         )
44
```

Annexe: Code

```
45  def to_json(self):
46      """Serializes the automata instance into a JSON string."""
47      data = {
48          "states": sorted(list(self.states)),
49          "alphabet": sorted(list(self.alphabet)),
50          "transition": self.transition,
51          "start_state": self.start_state,
52          "accept_states": sorted(list(self.accept_states))
53      }
54      return json.dumps(data)
55
56  def generate_trace(self, input_string):
57      trace = {'i': [], 's': [], 'c': []}
58      current_state = self.start_state
59      i = 0
60      for symbol in input_string:
61          if symbol not in self.alphabet:
62              return False
63          trace['i'].append(i)
64          i+=1
65          trace['s'].append(current_state)
66          trace['c'].append(symbol)
67          current_state = self.transition[current_state][symbol]
68
69      return trace
```



Annexe: Code

```
#word = open("text.txt", "r").read()
word = "I think STARKS are fun to learn"
```

```
import pandas as pd
from itertools import product
import time
from poseidon import Poseidon, ARC, generate_cauchy_matrix
from field import FieldElement
import numpy as np
from polynomial import Polynomial, interpolate_poly
from channel import Channel
from merkle import MerkleTree, verify_decommitment
from functools import reduce
from constants import N_QUERY, r_f, r_p, blowup_factor, T
import constants
from utils import Automata, primes
from tqdm import tqdm
```

```
start_all = time.time()
```

Annexe: Code

```
import time
import math

def print_prover_success_banner(start_time, channel, POSEIDON_TRACE_EVAL_LENGTH, STATE_EVAL_LENGTH):
    # 1. Calculations
    duration = time.time() - start_time

    # Proof Size
    proof_size_bytes = sum(len(entry.encode('utf-8')) for entry in channel.proof)
    proof_size_kb = proof_size_bytes / 1024

    # Theoretical Soundness (The claim you are making)
    # Max degree is determined by the Poseidon constraint (degree 5)
    max_degree = (POSEIDON_TRACE_EVAL_LENGTH - 1) * 5
    rho = max_degree / STATE_EVAL_LENGTH
    p_cheat = rho ** N_QUERY
    security_bits = -math.log2(p_cheat) if p_cheat > 0 else 0

    # Expansion Factor (Blowup)
    blowup = STATE_EVAL_LENGTH / POSEIDON_TRACE_EVAL_LENGTH

    # 2. Colors & Art
    BLUE = "\033[94m"
    GREEN = "\033[92m"
    YELLOW = "\033[93m"
    MAGENTA = "\033[95m"
    CYAN = "\033[96m"
    RESET = "\033[0m"
    BOLD = "\033[1m"

    ascii_art = f"""
{BLUE}{BOLD}

PROOF GEN

{RESET}"""

    print(ascii_art)
    print(f"{BOLD}Status:{RESET} {BLUE}PROOF GENERATED SUCCESSFULLY{RESET}")
    print(f"{BOLD} * 60")
    print(f"{BOLD}Computational Effort:{RESET}")
    print(f"⌚ {YELLOW}Generation Time:{RESET} {duration:.4f} seconds")
    print(f"📏 {CYAN}Trace Length:{RESET} {POSEIDON_TRACE_EVAL_LENGTH} steps")
    print(f"📦 {CYAN}Blowup Factor:{RESET} {blowup:.1f}x (Domain size: {STATE_EVAL_LENGTH})")
    print(f"{BOLD} * 60")
    print(f"{BOLD}Proof Artifacts:{RESET}")
    print(f"📄 {GREEN}Proof Size:{RESET} {proof_size_kb:.2f} KB")
    print(f"🔒 {MAGENTA}Claimed Security:{RESET} {security_bits:.1f} bits (P(Cheat) = {p_cheat:.1e})")
    print(f"{BOLD} * 60")
```

File display

Annexe: Code

```
automata = Automata.from_json(open("automata.json", "r").read())
trace = automata.generate_trace(word)
```

```
mds = generate_cauchy_matrix(T)
```

```
hash_input = [FieldElement(x) for x in np.pad([automata.character_map[x] for x in word], (0, (-len(word)) % (T-1)))]
```

```
phash, ptrace = Poseidon().hash(hash_input)
```

```
X = Polynomial.X()
```

```
def next_power_of_two(n: int) -> int:
    if n <= 0:
        return 1
    # If n is already a power of two, return n
    if (n & (n - 1)) == 0:
        return n
    power = 1
    while power < n:
        power <<= 1
    return power
```

```
def hash_input_index(i,j):
    if (i % (r+r_p) == 0 and j < 7 and i//((r_f+r_p) < len(hash_input)//7):
        return hash_input[7*(i//((r_f+r_p))+j)]
    else:
        return FieldElement(0)
```

```
ptrace_domain_size = next_power_of_two(len(ptrace))
gp = FieldElement.generator() ** (3*2**30/(ptrace_domain_size))
x = [gp**i for i in range(len(trace['i']))]
z = [gp**i for i in range(len(ptrace))]
```

File display

```
state_polynomial = interpolate_poly(x, [FieldElement(automata.state_map[s]) for s in trace['s']])
character_polynomial = interpolate_poly(x, [FieldElement(automata.character_map[c]) for c in trace['c']])
transi_x = [FieldElement(automata.state_map[s]*automata.character_map[c]) for s,c in product(automata.states, automata.alphabet)]
transition_polynomial = interpolate_poly(transi_x, [FieldElement(automata.state_map[automata.transition[s][c]]) for (s,c) in product(automata.states, a
step_polynomial = state_polynomial(gp*X) - transition_polynomial(state_polynomial(X)*character_polynomial(X))
ptrace_polynomials = [interpolate_poly(z, [row[i] for row in ptrace]) for i in range(0, len(ptrace[0]))]
arc polys = [interpolate_poly(z, [FieldElement(ARC[j]*(r_f+r_p)[i]) for j in range(len(ptrace[0]))] for i in range(len(ARC[0]))]
hash_input_poly = interpolate_poly(z, [hash_input_index(i,j) for i in range(len(z))] for j in range(len(ptrace[0]))]
```

Annexe: Code

```
w = FieldElement.generator()
h = FieldElement.generator() ** (3*2**30/(ptrace_domain_size*blowup_factor))
H = [h**i for i in range(ptrace_domain_size*blowup_factor)]
eval_domain = [w*h for h in H]

state_eval = [state_polynomial(d) for d in tqdm(eval_domain)]
character_eval = [character_polynomial(d) for d in tqdm(eval_domain)]
#transition_eval = [transition_polynomial(state*character) for state, character in tqdm(product(state_eval, character_eval))]
transition_eval = [transition_polynomial(state*character) for state, character in zip(state_eval, character_eval)]
ptrace_evals = [[p(x) for x in tqdm(eval_domain)] for p in ptrace_polynomials]
hash_evals = [[p(x) for x in tqdm(eval_domain)] for p in hash_input_poly]
arc_polys_eval = [[p(x) for x in tqdm(eval_domain)] for p in arc_polys]
```

Annexe: Code

```
state_merkle = MerkleTree(state_eval)
character_merkle = MerkleTree(character_eval)
transition_merkle = MerkleTree(transition_eval)
ptrace_merkle = MerkleTree([e for p in ptrace_evals for e in p])
hash_merkle = MerkleTree([e for p in hash_evals for e in p])
arc_merkle = MerkleTree([e for p in arc_polys_eval for e in p])
```

```
initial_constraint_p = (state_polynomial - automata.state_map['q0'])/(X-1)
Z_G = Polynomial([FieldElement.one()])
for i in range(len(x)):
    Z_G = Z_G * (X-x[i])
ap = Polynomial([FieldElement.one()])
for c in automata.alphabet:
    ap = ap * (character_polynomial - automata.character_map[c])
character_constraint_p = ap/Z_G
ap = Polynomial([FieldElement.one()])
for f in automata.accept_states:
    ap = ap * (state_polynomial - automata.state_map[f])
accepting_constraint_p = ap/(X-x[-1])
step_constraint_p = step_polynomial/(Z_G/(X-x[-1]))
poseidon_full_round_polynomials = nds@([ptrace_polynomials[j](X) + hash_input_poly[j](X) + arc_polys[j](X)]*5 for j in range(T))
poseidon_partial_round_polynomials = nds@([ptrace_polynomials[j](X) + hash_input_poly[j](X) + arc_polys[j](X)]*5 if j == 0 else 1) for j in range(T))
X_G = Polynomial([FieldElement.one()])
Z_G = Polynomial([FieldElement.one()])
for i in range(len(ptrace)-1):
    r = 1%(r_f+r_p)
    if(r < r_f/2 or r >= r_f/2+r_p):
        X_G = X_G * (X-z[i])
    else:
        Z_G = Z_G * (X-z[i])
poseidon_constraint_full_round = [(ptrace_polynomials[j](gp*X) - poseidon_full_round_polynomials[j](X))/X_G for j in range(len(ptrace_polynomials))]
poseidon_constraint_partial_round = [(ptrace_polynomials[j](gp*X) - poseidon_partial_round_polynomials[j](X))/Z_G for j in range(len(ptrace_polynomials))]
poseidon_constraints_initial = [p/(X-1) for p in ptrace_polynomials]
poseidon_output_constraint = (ptrace_polynomials[0](X)-phash[0])/(X-z[-1])
```

File display

Annexe: Code

```
channel = Channel()
channel.send(str(ptrace_domain_size))
channel.send(str(len(ptrace)))
channel.send(str(len(eval_domain)))
channel.send(str(len(trace['i'])))
channel.send(str(phash[0]))
channel.send(state_merkle.root)
channel.send(caracter_merkle.root)
channel.send(transition_merkle.root)
channel.send(ptrace_merkle.root)
channel.send(hash_merkle.root)
channel.send(arc_merkle.root)
```

```
ps = [poseidon_output_constraint, initial_constraint_p, caracer_constraint_p, accepting_constraint_p, step_constraint_p, *poseidon_constraints_initial
alphas = [channel.receive_random_field_element() for _ in range(len(ps))]
cp = reduce(lambda x, y: x*y, map(lambda a, p: a*p, alphas, ps), FieldElement(0))
```

```
cp_eval = [cp(x) for x in eval_domain]
cp_merkle = MerkleTree(cp_eval)
channel.send(cp_merkle.root)
```

```
def fold_domain(domain):
    return [x**2 for x in domain[:len(domain)//2]]
```

```
def fold(p, beta):
    odd = p.poly[1::2]
    even = p.poly[::2]
    o = Polynomial(odd)
    e = Polynomial(even)
    return e + beta*o
```

```
def next_fri_layer(poly, domain, beta):
    next_poly = fold(poly, beta)
    next_domain = fold_domain(domain)
    next_layer = [next_poly(x) for x in next_domain]
    return next_poly, next_domain, next_layer
```

File display

```
cp.degree(), len(cp_eval)
```

Annexe: Code

```
def FriCommit(cp, domain, cp_eval, cp_merkle, channel):
    fri_polys = [cp]
    fri_domains = [domain]
    fri_layers = [cp_eval]
    fri_merkles = [cp_merkle]
    betas = []
    i = 0
    while fri_polys[-1].degree() > 0:
        beta = channel.receive_random_field_element()
        betas.append(beta)
        next_poly, next_domain, next_layer = next_fri_layer(fri_polys[-1], fri_domains[-1], beta)
        fri_polys.append(next_poly)
        fri_domains.append(next_domain)
        fri_layers.append(next_layer)
        fri_merkles.append(MerkleTree(next_layer))
        channel.send(fri_merkles[-1].root)
        i+=1
    channel.send("FINISHED_FRI")
    channel.send(str(fri_polys[-1].poly[0]))
    return fri_polys, fri_domains, fri_layers, fri_merkles, betas
```

```
fri_polys, fri_domains, fri_layers, fri_merkles, betas = FriCommit(cp, eval_domain, cp_eval, cp_merkle, channel)
```

```
len(fri_layers[-1])
```

```
def decommit_on_fri_layers(idx, channel):
    prev_idx = None
    prev_sibidx = None
    p_idx = None
    i = 0
    for layer, merkle in zip(fri_layers[:-1], fri_merkles[:-1]):
        length = len(layer)
        idx = idx % length
        sib_idx = (idx + length // 2) % length
        channel.send(str(layer[idx]))
        channel.send(str(merkle.get_authentication_path(idx)))
        channel.send(str(layer[sib_idx]))
        channel.send(str(merkle.get_authentication_path(sib_idx)))
    channel.send(str(fri_layers[-1][0]))
```

Annexe: Code

```
def decommit_on_query(idx, channel):
    assert idx + blowup_factor < len(eval_domain), f'query index: {idx} is out of range. Length of layer: {len(eval_domain)}.'
    channel.send(str(state_eval[idx%len(eval_domain)])) # f(x).
    channel.send(str(state_merkle.get_authentication_path(idx%len(eval_domain)))) # auth path for f(x).
    assert(state_eval[(idx + blowup_factor)%len(eval_domain)] == state_polynomial(gp*eval_domain[idx%len(eval_domain)]))
    channel.send(str(state_eval[(idx + blowup_factor)%len(eval_domain)])) # f(gx).
    channel.send(str(state_merkle.get_authentication_path((idx + blowup_factor)%len(eval_domain)))) # auth path for f(gx).
    assert (idx + blowup_factor)%len(eval_domain) < len(character_eval), f'query index: {idx%len(eval_domain)} is out of range. Length of layer: {len(ca
    channel.send(str(character_eval[idx%len(eval_domain)])) # f(x).
    channel.send(str(character_merkle.get_authentication_path(idx%len(eval_domain)))) # auth path for f(x).
    assert (idx + blowup_factor)%len(eval_domain) < len(transition_eval), f'query index: {idx%len(eval_domain)} is out of range. Length of layer: {len(
    transi_x = transition_polynomial(character_eval[idx%len(eval_domain)]*state_eval[idx%len(eval_domain)])
    channel.send(str(transi_x)) # f(x).
    transi_idx = (len(eval_domain)*idx+idx)%len(eval_domain)
    channel.send(str(transition_merkle.get_authentication_path(transi_idx))) # auth path for f(x).

    #### POSEIDON ####
    for row in range(len(ptrace[0])):
        trace_x = ptrace_evals[row][idx%len(eval_domain)]
        channel.send(str(trace_x))
        channel.send(str(ptrace_merkle.get_authentication_path((len(eval_domain)*row+(idx%len(eval_domain)))))
        assert(ptrace_polynomials[row][eval_domain[idx%len(eval_domain)]] == trace_x)

        next_trace_x = ptrace_evals[row][(idx+blowup_factor)%len(eval_domain)]
        channel.send(str(next_trace_x))
        channel.send(str(ptrace_merkle.get_authentication_path((len(eval_domain)*row+((idx+blowup_factor)%len(eval_domain)))))
        assert(ptrace_polynomials[row][gp*eval_domain[idx%len(eval_domain)]] == next_trace_x)

        hash_x = hash_evals[row][idx%len(eval_domain)]
        channel.send(str(hash_x))
        channel.send(str(hash_merkle.get_authentication_path((len(eval_domain)*row+(idx%len(eval_domain)))))

        arc_x = arc_polys_eval[row][idx%len(eval_domain)]
        channel.send(str(arc_x))
        channel.send(str(arc_merkle.get_authentication_path((len(eval_domain)*row+(idx%len(eval_domain)))))

    decommit_on_fri_layers(idx, channel)
```

```
def decommit_fri(channel):
    for query in range(N_QUERY):
        decommit_on_query(channel.receive_random_int(0, len(eval_domain)), channel)
```

decommit_fri(channel)

File display

Annexe: Code

```
proof_file = open("proof.txt", "w")  
proof_file.write(str(channel.proof))
```

```
print_prover_success_banner(start_all, channel, len(ptrace), len(state_eval))
```

Annexe: Code

```
import ast
import constants
from constants import blowup_factor
from channel import Channel
from field import FieldElement
from merkle import verify_decommitment
from poseidon import generate_cauchy_matrix
from functools import reduce
from utils import Automata
```

Annexe: Code

```
import time
import sys

def print_success_banner(start_time, channel, EVAL_DOMAIN_LENGTH, POSEIDON_TRACE_EVAL_LENGTH):
    # 1. Calculate Duration
    duration = time.time() - start_time

    # 2. Calculate Proof Size
    # We sum the bytes of all strings sent in the proof
    proof_size_bytes = sum(len(entry.encode('utf-8')) for entry in channel.proof)
    proof_size_kb = proof_size_bytes / 1024

    # 3. ASCII Art & Colors
    GREEN = "\033[92m"
    CYAN = "\033[96m"
    YELLOW = "\033[93m"
    RESET = "\033[0m"
    BOLD = "\033[1m"

    ascii_art = f"""
{GREEN}{BOLD}
PROOF VALID
{RESET}"""

    print(ascii_art)
    print(f"{BOLD}Status:{RESET} {GREEN}VERIFICATION SUCCESSFUL{RESET}")
    print(f"{BOLD}Metrics:{RESET}")
    print(f"⌚ {CYAN}Time Taken:{RESET} {duration:.4f} seconds")
    print(f"📄 {YELLOW}Proof Size:{RESET} {proof_size_kb:.2f} KB")
    print(f"🔍 {CYAN}Queries:{RESET} {constants.N_QUERY}")
    max_constraint_degree = (POSEIDON_TRACE_EVAL_LENGTH - 1) * 5

    # 2. Calculate Probability of Cheating (Soundness Error)
    # Rho = Degree / Domain Size
    rho = max_constraint_degree / EVAL_DOMAIN_LENGTH
    p_cheat = rho ** constants.N_QUERY

    # 3. Print Code
    print(f"🎲 {CYAN}P(Cheating):{RESET} {p_cheat:.8f} (approx. 1 in {int(1/p_cheat)})")
    print(f"{BOLD}Optional: Contextualize for the Jury")
    print(f"{BOLD}Interpretation:{RESET}")
    if proof_size_kb < 200:
        print("✅ Succinct Proof (Very small size!)")
    else:
        print("⚠️ Large Proof (Consider checking trace/blowup)")
```

Annexe: Code

```
proof_file = open("proof.txt", "r")
proof = ast.literal_eval(proof_file.read())
automata = Automata.from_json(open("automata.json", "r").read())
```

```
def parse_proof(method, parse = None):
    global v_current_channel_idx
    global v_proof
    #print("channel", proof[current_channel_idx], "method", method)
    v_current_channel_idx+=1
    if parse == None:
        return v_proof[v_current_channel_idx-1][len(method)+1:]
    else:
        return parse[v_proof[v_current_channel_idx-1][len(method)+1:]])
```

Annexe: Code

```
channel = Channel(proof)
v_proof = channel.proof
v_c = Channel()
v_current_channel_idx = 0
PTRACE_DOMAIN_SIZE=parse_proof("send", lambda x: int(x))
POSEIDON_TRACE_EVAL_LENGTH=parse_proof("send", lambda x: int(x))
EVAL_DOMAIN_LENGTH=parse_proof("send", lambda x: int(x))
AUTOMATA_TRACE_LENGTH=parse_proof("send", lambda x: int(x))
v_hash = v_proof[v_current_channel_idx][5:]
v_c.send(v_hash)
v_current_channel_idx+=1
v_state_merkle = v_proof[v_current_channel_idx][5:]
v_c.send(v_state_merkle)
v_current_channel_idx+=1
v_caracter_merkle = v_proof[v_current_channel_idx][5:]
v_c.send(v_caracter_merkle)
v_current_channel_idx+=1
v_transition_merkle = v_proof[v_current_channel_idx][5:]
v_c.send(v_transition_merkle)
v_current_channel_idx+=1
v_ptrace_merkle = v_proof[v_current_channel_idx][5:]
v_c.send(v_ptrace_merkle)
v_current_channel_idx+=1
v_hash_merkle = v_proof[v_current_channel_idx][5:]
v_c.send(v_hash_merkle)
v_current_channel_idx+=1
v_arc_merkle = v_proof[v_current_channel_idx][5:]
v_c.send(v_arc_merkle)
v_current_channel_idx+=1
v_alphas = []
for i in range(constants.CONSTRAINTS_LENGTH):
    v_a = FieldElement(int(v_proof[v_current_channel_idx][len('receive_random_field_element:')]))
    v_current_channel_idx+=1
    v_alphas.append(v_a)
#a0 = FieldElement(int(proof[current_channel_idx][len('receive_random_field_element:')]))
#assert(c.receive_random_field_element() == a0)
#current_channel_idx+=1
#a1 = FieldElement(int(proof[current_channel_idx][len('receive_random_field_element:')]))
#assert(c.receive_random_field_element() == a1)
#current_channel_idx+=1
#a2 = FieldElement(int(proof[current_channel_idx][len('receive_random_field_element:')]))
#assert(c.receive_random_field_element() == a2)
#current_channel_idx+=1
#a3 = FieldElement(int(proof[current_channel_idx][len('receive_random_field_element:')]))
#assert(c.receive_random_field_element() == a3)
#current_channel_idx+=1
#a4 = FieldElement(int(proof[current_channel_idx][len('receive_random_field_element:')]))
#assert(c.receive_random_field_element() == a4)
#current_channel_idx+=1
cp_merkle_root = v_proof[v_current_channel_idx][len('send:'):]
v_current_channel_idx+=1
```

File display

Annexe: Code

```
w = FieldElement.generator()
h = FieldElement.generator() ** (3*2**30/(PTRACE_DOMAIN_SIZE*blowup_factor))
gp = FieldElement.generator() ** (3*2**30/(PTRACE_DOMAIN_SIZE))

#def x(i):
#    return gp**i

def x(i):
    return gp**(i%AUTOMATA_TRACE_LENGTH)

def z(i):
    return gp**(i%POSEIDON_TRACE_EVAL_LENGTH)

#x = [gp**i for i in range(AUTOMATA_TRACE_LENGTH)]
#z = [gp**i for i in range(POSEIDON_TRACE_EVAL_LENGTH)]
mds = generate_cauchy_matrix(constants.T)

def verify_FRI_commitment():
    global v_proof
    global v_current_channel_idx
    v_betas = []
    v_roots = [cp_merkle_root]
    v_l = 0
    while v_proof[v_current_channel_idx] != "send:FINISHED_FRI":
        v_betas.append(FieldElement(int(v_proof[v_current_channel_idx][len('receive_random_field_element:')]))
        v_current_channel_idx+=1
        v_roots.append(v_proof[v_current_channel_idx][len('send:')])
        v_current_channel_idx+=1
        v_l+=1
    v_current_channel_idx+=1
    v_poly0 = int(v_proof[v_current_channel_idx][len('send:')])
    v_current_channel_idx+=1
    return v_betas, v_roots, v_poly0, v_l

v_betas, v_roots, v_poly0, v_l = verify_FRI_commitment()

print(v_l, 2**10)
```

Annexe: Code

```
def verify_query_decommitment():
    global current_channel_idx
    for query in range(constants.N_QUERY):
        # r = int(proof[current_channel_idx][len('receive_random_field_element:')])
        r = parse_proof("receive_random_int", int)
        # current_channel_idx += 1
        state_x = parse_proof("send", lambda x: FieldElement(int(x)))
        # state_x = FieldElement(int(proof[current_channel_idx][len('send:')]))
        # current_channel_idx += 1
        state_x_path = parse_proof("send", lambda x: ast.literal_eval(x))
        state_x2 = parse_proof("send", lambda x: FieldElement(int(x)))
        # state_x2 = FieldElement(int(proof[current_channel_idx][len('send:')]))
        # current_channel_idx += 1
        state_x2_path = parse_proof("send", lambda x: ast.literal_eval(x))
        # state_x2_path = ast.literal_eval(proof[current_channel_idx][len('send:')])
        # current_channel_idx += 1
        assert(verify_decommitment(r, state_x, state_x_path, v_state_merkle))
        assert(verify_decommitment((r+blowup_factor), state_x2, state_x2_path, v_state_merkle))

        # character_x = FieldElement(int(proof[current_channel_idx][len('send:')]))
        # current_channel_idx += 1
        character_x = parse_proof("send", lambda x: FieldElement(int(x)))
        character_x_path = parse_proof("send", lambda x: ast.literal_eval(x))
        # character_x_path = proof[current_channel_idx][len('send:'):]
        # current_channel_idx += 1
        assert(verify_decommitment(r, character_x, character_x_path, v_character_merkle))

        transition_x = parse_proof("send", lambda x: FieldElement(int(x)))
        transition_x_path = parse_proof("send", lambda x: ast.literal_eval(x))
        assert(verify_decommitment((EVAL_DOMAIN_LENGTH+r), transition_x, transition_x_path, v_transition_merkle))

    poseidon_x = []
    poseidon_path = []
    poseidon_next_x = []
    poseidon_next_path = []
    hash_x = []
    hash_path = []
    arc_x = []
    arc_path = []

    for i in range(constants.T):
        poseidon_x.append(parse_proof("send", lambda x: FieldElement(int(x))))
        poseidon_path.append(parse_proof("send", lambda x: ast.literal_eval(x)))
        assert(verify_decommitment((i+EVAL_DOMAIN_LENGTH + r), poseidon_x[i], poseidon_path[i], v_pttrace_merkle))

        poseidon_next_x.append(parse_proof("send", lambda x: FieldElement(int(x))))
        poseidon_next_path.append(parse_proof("send", lambda x: ast.literal_eval(x)))
        assert(verify_decommitment((i+EVAL_DOMAIN_LENGTH + ((r + blowup_factor))), poseidon_next_x[i], poseidon_next_path[i], v_pttrace_merkle))

        hash_x.append(parse_proof("send", lambda x: FieldElement(int(x))))
        hash_path.append(parse_proof("send", lambda x: ast.literal_eval(x)))
        assert(verify_decommitment((i+EVAL_DOMAIN_LENGTH + r), hash_x[i], hash_path[i], v_hash_merkle))

        arc_x.append(parse_proof("send", lambda x: FieldElement(int(x))))
        arc_path.append(parse_proof("send", lambda x: ast.literal_eval(x)))
        assert(verify_decommitment((i+EVAL_DOMAIN_LENGTH + r), arc_x[i], arc_path[i], v_arc_merkle))
```

File display

Annexe: Code

```
#transition_x = FieldElement(int(proof[current_channel_idx][len('send':)]))
#current_channel_idx+=1
#transition_x_path = proof[current_channel_idx][len('send':):]
#current_channel_idx+=1

layers = []
layer_paths = []

#length = 2**(v_l+1)
length = EVAL_DOMAIN_LENGTH
cps = []

for j in range(0,1):
    idx = r % length
    sib_idx = (r + length // 2) % length
    layer_x = parse_proof("send", lambda x: FieldElement(int(x)))
    #layer_x = int(proof[current_channel_idx][len('send':):])
    #current_channel_idx+=1
    layer_x_path = parse_proof("send", lambda x: ast.literal_eval(x))
    assert(verify_decommitment(idx, layer_x, layer_x_path, v_roots[j]))
    #layer_x_path = proof[current_channel_idx][len('send':):]
    #current_channel_idx+=1
    layer_sibx = parse_proof("send", lambda x: FieldElement(int(x)))
    #layer_sibx = int(proof[current_channel_idx][len('send':):])
    #current_channel_idx+=1
    #layer_sibx_path = proof[current_channel_idx][len('send':):]
    layer_sibx_path = parse_proof("send", lambda x: ast.literal_eval(x))
    assert(verify_decommitment(sib_idx, layer_sibx, layer_sibx_path, v_roots[j]))
    e = (w*h*idx)*(2**j)

    g_x = (layer_x + layer_sibx)/FieldElement(2)
    h_x = (layer_x - layer_sibx)/(2*e)
    cp_ip1 = g_x + v_betas[j]*h_x
    cps.append(cp_ip1)

if j == 0:
    initial_constraint = (state_x-automata.state_map['q0'])/(e-1)
    Z_G = FieldElement(1)
    for i in range(AUTOMATA_TRACE_LENGTH):
        Z_G = Z_G * (e-x[i])
        ap = FieldElement(1)
        for c in automata.alphabet:
            ap = ap * (character_x - automata.character_map[c])
        character_constraint = ap/Z_G
        ap = FieldElement(1)
        for f in automata.accept_states:
            ap = ap * (state_x - automata.state_map[f])
        accepting_constraint = ap/(e-x[-1])
        step_constraint = (state_x2 - transition_x)/(Z_G/(e-x[-1]))
        X_G = FieldElement(1)
        Y_G = FieldElement(1)
```

Annexe: Code

```
for i in range(POSEIDON_TRACE_EVAL_LENGTH-1):
    round_x = i%(constants.r_f+constants.r_p)
    if round_x < constants.r_f/2 or round_x >= constants.r_f/2+constants.r_p:
        X_G = X_G * (e-z(i))
    else:
        Y_G = Y_G * (e-z(i))
    v_poseidon_full_round = mds@[(poseidon_x[j] + hash_x[j] + arc_x[j])**5 for j in range(constants.T)]
    v_poseidon_partial_round = mds@[(poseidon_x[j] + hash_x[j] + arc_x[j])**5 if j == 0 else 1 for j in range(constants.T)]
    poseidon_full_round_cr = [(poseidon_next_x[j] - v_poseidon_full_round[j])/X_G for j in range(constants.T)]
    poseidon_partial_round_cr = [(poseidon_next_x[j] - v_poseidon_partial_round[j])/Y_G for j in range(constants.T)]
    poseidon_initial_cr = [poseidon_x[j]/(e-1) for j in range(constants.T)]
    poseidon_output_cr = [poseidon_x[0]-FieldElement(int(v_hash))/(e-z(-1))]

    v_ps = [poseidon_output_cr, initial_constraint, character_constraint, accepting_constraint, step_constraint, *poseidon_initial_cr, *poseidon_output_cr]
    cp_e = reduce(lambda x, y: x*y, map(lambda a, p: a*p, v_alphas, v_ps), FieldElement(0))
    assert(cp_e == layer_x)
else:
    assert(cps[j-1] == layer_x)

layers.append((layer_x, layer_sibx))
layer_paths.append((layer_x_path, layer_sibx_path))
length //= 2

last_layer_x = parse_proof("send", int)
assert(cps[-1] == last_layer_x)
```

```
verify_query_decommitment(v_l)
```

```
print_success_banner(verification_start, channel, EVAL_DOMAIN_LENGTH, POSEIDON_TRACE_EVAL_LENGTH)
```

PROOF VALID

Status: VERIFICATION SUCCESSFUL

Metrics:

🕒 Time Taken:	0.2546 seconds
📄 Proof Size:	771.66 KB
🔍 Queries:	12
🎲 P(Cheating):	0.00000000 (approx. 1 in 18514789709398876160)

Interpretation:

⚠️ Large Proof (Consider checking trace/blowup)